

Well-Architected: los seis pilares

Mirar el conjunto, no solo las piezas

Cada sesión del módulo ha resuelto un problema concreto de Escaparate. Las decisiones se han tomado de una en una y nadie ha mirado el conjunto.

 Dónde vive

 Quién puede llegar a él

 Qué pasa cuando algo falla

 Cuánto cuesta

 Cómo se reconstruye

Hoy: la arquitectura entera, con un marco profesional

Revisas la arquitectura de Escaparate, tal como quedó al terminar la Actividad 5.2, con el marco que usan los equipos de arquitectura para decidir si un sistema está bien construido, no solo si funciona.

No abres AWS ni despliegas nada: partes de una ficha con todos los componentes de Escaparate y trabajas sobre el papel.

Funcionar no es lo mismo que estar bien construido

Un catálogo puede cargar sus productos sin un solo error y, a la vez, esconder estos tres problemas. Nada de eso se nota mientras todo va bien: se nota el día que algo falla.



Una sola zona

La base de datos vive en una única zona de disponibilidad.



Puerto abierto

Un puerto de administración abierto a todo internet.



Sin avisos

Alarmas que nadie recibe.

Auditoría de arquitectura

Recorrer un sistema con una lista fija de preguntas, para no depender de que se te ocurra qué mirar.

El marco Well-Architected: seis pilares

La lista más usada es el marco Well-Architected de AWS: buenas prácticas recogidas de las revisiones de miles de sistemas reales y organizadas en seis pilares —seis ángulos desde los que se mira cualquier arquitectura—, cada uno con su pregunta.

Excelencia operativa

¿Puedes ver lo que pasa y mejorar el sistema con confianza?

Seguridad

¿Están protegidos los datos y los accesos?

Fiabilidad

¿Se recupera el sistema de un fallo sin intervención?

Eficiencia del rendimiento

¿Usas los recursos adecuados para la carga real?

Optimización de costes

¿Gastas lo que necesitas, ni más ni menos?

Sostenibilidad

¿Minimizas la energía y los recursos que consume lo que tienes encendido?

Los seis pilares: lo que ya has practicado

Pilar	Pregunta que responde	Ya lo has practicado
Excelencia operativa	¿Puedes ver lo que pasa y mejorar con confianza?	Dashboard y alarmas (Tema 5), infraestructura en código (Tema 6)
Seguridad	¿Están protegidos los datos y los accesos?	Subredes privadas y grupos de seguridad (Temas 2 y 3), mínimo privilegio (Tema 5)
Fiabilidad	¿Se recupera de un fallo sin intervención?	Grupo de escalado que repone instancias (Tema 4), Multi-AZ (Tema 3)
Eficiencia del rendimiento	¿Usas los recursos adecuados para la carga real?	Tipo de instancia y escalado por CPU (Temas 2 y 4)
Optimización de costes	¿Gastas lo que necesitas, ni más ni menos?	Calculadora y modelos de compra (Tema 5)
Sostenibilidad	¿Minimizas la energía y los recursos de lo que tienes encendido?	Capacidad que se apaga sola cuando no hace falta (Temas 4 y 6)

No es una lista nueva: es una forma de mirar

Nada nuevo que aprender

Cada pilar corresponde a algo que ya has practicado. El marco te pide mirar una arquitectura con las seis preguntas encima, una a una, sin saltarte ninguna.

Sostenibilidad: qué te toca a ti

AWS se encarga de la eficiencia de sus centros de datos. A ti te toca no mantener encendida capacidad que nadie usa.

Las seis preguntas, una a una

 Operativa

 Seguridad

 Fiabilidad


Rendimiento

 Costes


Sostenibilidad

Qué se comprueba en cada pilar (1/3)

Pilar	Qué se comprueba	Cómo se suele corregir	Dónde lo has visto
Excelencia operativa	¿Cada alarma avisa a alguien? Una alarma que no notifica a nadie se dispara igual, pero nadie se entera. ¿Los registros caducan o se acumulan para siempre?	Añadir una notificación a la alarma (por ejemplo, un correo) y fijar una retención a los registros.	Temas 5 y 6
Seguridad	¿Hay puertos de administración abiertos a todo internet? ¿Todo va cifrado? ¿Las piezas con datos están en subredes privadas? ¿Cada identidad tiene solo los permisos que necesita, y cada persona la suya?	Limitar el SSH a una IP o entrar por un bastión; HTTPS con certificado; subredes privadas (con NAT si tienen que salir); un rol con solo lo necesario y un usuario por persona.	Temas 2, 3, 4 y 5

Qué se comprueba en cada pilar (2/3)

Pilar	Qué se comprueba	Cómo se suele corregir	Dónde lo has visto
Fiabilidad	¿Alguna pieza vive en una sola zona? ¿Las copias están fuera de lo que puede fallar junto con el original (misma máquina, zona o región)? ¿Algo se repone solo?	Activar Multi-AZ; guardar las copias en otra máquina y, si el riesgo lo justifica, en otra región; un grupo de escalado que reponga lo que falle.	Temas 3, 4 y 7
Eficiencia del rendimiento	¿La capacidad se ajusta a la carga? ¿El contenido que se repite se sirve desde una caché cercana al usuario?	Escalar según la carga que se mide; una red de distribución de contenido (CDN) delante del contenido que se repite.	Tema 4

Qué se comprueba en cada pilar (3/3)

Pilar	Qué se comprueba	Cómo se suele corregir	Dónde lo has visto
Optimización de costes	¿Hay algo encendido que ya no se usa? Un recurso olvidado sigue costando. ¿Alguna pieza es más grande de lo que la carga necesita?	Borrar lo que no se usa; ajustar el tamaño a lo que se mide; elegir el modelo de compra que encaje.	Tema 5
Sostenibilidad	¿Hay capacidad encendida a todas horas que solo se usa una parte del día?	Reducir o apagar la capacidad cuando no hace falta, siempre que el servicio lo permita.	Temas 4 y 6

Cómo se ve un hallazgo: un ejemplo completo

Hallazgo

Un problema concreto de una arquitectura, anotado con dos datos:

Impacto

Qué pasa si no se corrige.

Esfuerzo

Cuánto cuesta corregirlo.

El caso: la aplicación de préstamos de una biblioteca de instituto

Una sola instancia t3.large con IP pública que ejecuta la aplicación y la base de datos PostgreSQL. La copia de seguridad es un fichero que alguien copia a mano a su portátil cada mes. La contraseña de la base de datos está escrita en el fichero de configuración que se sube al repositorio. La única forma de enterarse de una caída es que un alumno avise al bibliotecario.

Cómo se ve un hallazgo (cont.)

Pilar	Hallazgo	Impacto	Esfuerzo
Fiabilidad	Una sola instancia con aplicación y base de datos: si falla, cae todo	Alto	Medio: base de datos gestionada
Fiabilidad	Copia manual mensual: se pueden perder hasta 30 días de préstamos	Alto	Bajo: copias automáticas diarias
Seguridad	Contraseña de la base de datos dentro del repositorio	Alto	Bajo: gestor de secretos y rotarla
Seguridad	Solo HTTP: las contraseñas de los alumnos viajan sin cifrar	Alto	Medio: certificado y HTTPS
Excelencia operativa	Nadie recibe un aviso cuando la aplicación deja de responder	Medio	Bajo: una alarma con notificación
Optimización de costes	Instancia grande encendida 24 horas para una biblioteca que abre 8	Bajo	Bajo: tamaño menor y apagado nocturno

Si tras revisar un pilar no encuentras nada, se anota «sin hallazgos» junto con lo que has comprobado: no se inventa un problema para rellenar la casilla.

Los pilares tiran en direcciones distintas

Casi ninguna mejora es gratis: lo que mejora un pilar suele empeorar otro.

Mejora	Qué pilar mejora	Qué pilar empeora, y por qué
Activar Multi-AZ en la base de datos	Fiabilidad	Costes: se paga una segunda instancia completa funcionando siempre
Cambiar a una instancia más grande	Rendimiento	Costes y sostenibilidad: más capacidad encendida de la que se usa
Reducir los permisos de un rol al mínimo	Seguridad	Excelencia operativa: cada acción nueva que falte hay que detectarla y añadirla
Poner una caché delante de las imágenes	Rendimiento y costes	Excelencia operativa: una pieza más que operar, y una imagen puede verse desactualizada mientras dura su TTL
Apagar las instancias fuera de horario	Costes y sostenibilidad	Fiabilidad: el sistema no responde mientras está apagado

Los pilares tiran en direcciones distintas (cont.)

Una auditoría no termina en «corrige todo»: termina en decidir qué pilar se prioriza en cada caso.

La misma mejora: apagar las instancias fuera de horario



Entorno de pruebas

Tiene sentido: ahorras coste y energía y nadie lo echa de menos.



Tienda que vende a cualquier hora

Es un error: el sistema no responde mientras está apagado.

La misma mejora puede ser correcta en un caso y un error en otro.

Disponibilidad: los nueves

La disponibilidad es el porcentaje del tiempo en que un sistema responde. Se habla de «nueves» porque cada nueve más reduce mucho la caída permitida.

99 %

3,65 días de caída
al año

99,9 %

8,76 horas de caída
al año

99,99 %

52,6 minutos de
caída al año

99,999 %

5,26 minutos de
caída al año

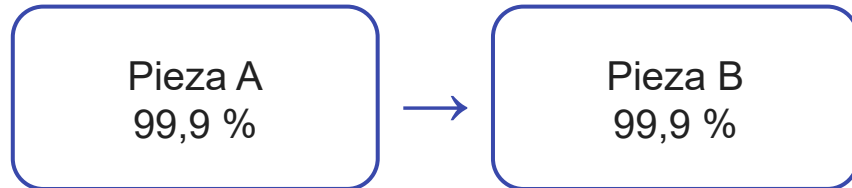
SLA (Service Level Agreement)

Cada servicio de AWS publica su propio compromiso de disponibilidad. Si AWS no lo cumple, devuelve una parte del importe.

Disponibilidad: piezas en serie y en paralelo

En serie

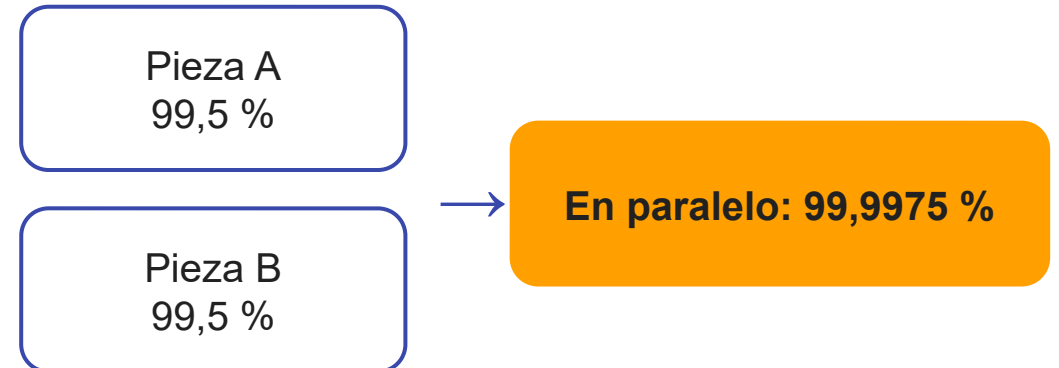
Para responder hacen falta todas las piezas: las disponibilidades se multiplican. Dos piezas de 99,9 % dan 99,8 %.



En serie: 99,8 %

En paralelo

Basta con que funcione una: el sistema solo cae si caen todas a la vez, es decir, se multiplican las probabilidades de fallo. Dos piezas de 99,5 % caen solo con probabilidad $0,005 \times 0,005$.



Una de las dos basta para que el sistema responda.

Disponibilidad de la biblioteca: el cálculo

Con la biblioteca del ejemplo anterior, y con cifras inventadas solo para el cálculo:

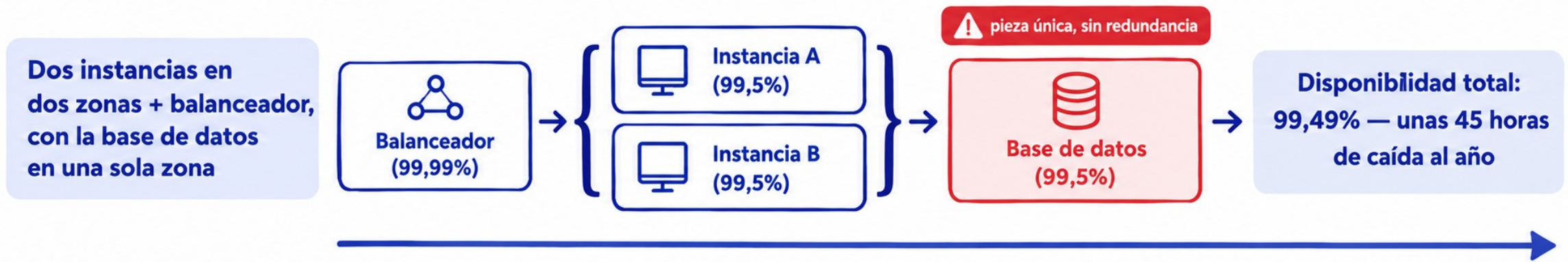
Configuración	Cálculo	Disponibilidad	Caída al año
Una instancia con aplicación y base de datos (99,5 %)	0,995	99,5 %	unas 44 horas
Dos instancias en dos zonas tras un balanceador (99,99 %)	$(1 - 0,005^2) \times 0,9999$	99,9875 %	unos 66 minutos
Lo anterior, pero con la base de datos aparte, en una sola zona (99,5 %)	$0,999875 \times 0,995$	99,49 %	unas 45 horas

La última fila es la que importa

Añadir instancias no ha servido de nada mientras la base de datos siga siendo una sola pieza.

El sistema es tan disponible como su eslabón más débil

El sistema es tan disponible como su eslabón más débil



Añadir instancias en paralelo (balanceador, A y B) no sirve de nada mientras la base de datos siga siendo una pieza única en serie. El sistema es tan disponible como esa pieza.

La redundancia no funciona tan bien como dice la fórmula

El cálculo en paralelo supone dos cosas.

Copias independientes

Se supone que las dos copias fallan de forma independiente.

Cambio instantáneo

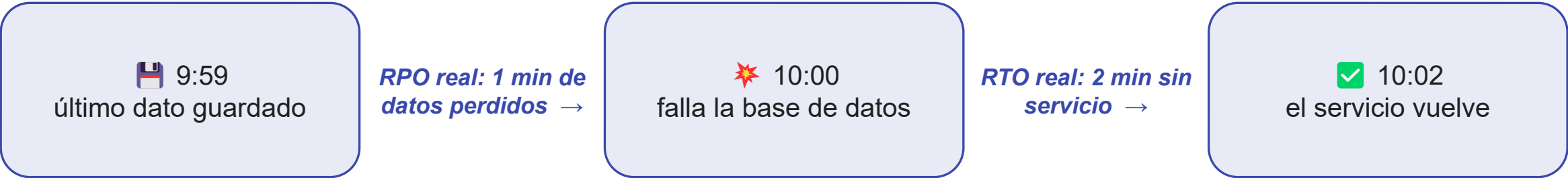
Se supone que el paso de una a otra es inmediato. En la práctica tarda: es tu RTO.

Hay fallos que afectan a las dos copias a la vez

Un error de configuración, un despliegue defectuoso o la caída de una región entera. Por eso la fórmula da un techo, no una garantía.

RTO, RPO y recuperación ante desastres

Métrica	Pregunta que responde	Ejemplo
RTO (Recovery Time Objective)	¿Cuánto tiempo puede estar el sistema caído antes de que sea inaceptable?	«El catálogo no puede estar más de 10 minutos sin responder»
RPO (Recovery Point Objective)	¿Cuántos datos recientes puedes permitirte perder?	«No podemos perder más de 5 minutos de pedidos»



El mismo incidente, dos preguntas distintas: Multi-AZ devuelve el servicio a las 10:02 (RTO de 2 minutos) y la copia tenía los datos hasta las 9:59 (RPO de 1 minuto).

RTO y RPO: los fija el negocio

No los fija quien construye el sistema

Es el dueño de una tienda quien sabe cuánto le cuesta una hora sin vender o un día de pedidos perdidos. Cuanto más exigentes son, más redundancia hay que pagar.



RTO

Cuánto tiempo caído es aceptable.



RPO

Cuántos datos recientes puedes perder.

Ninguna de las dos cifras te la da la otra, y las dos importan para decidir qué mecanismo de recuperación necesitas.

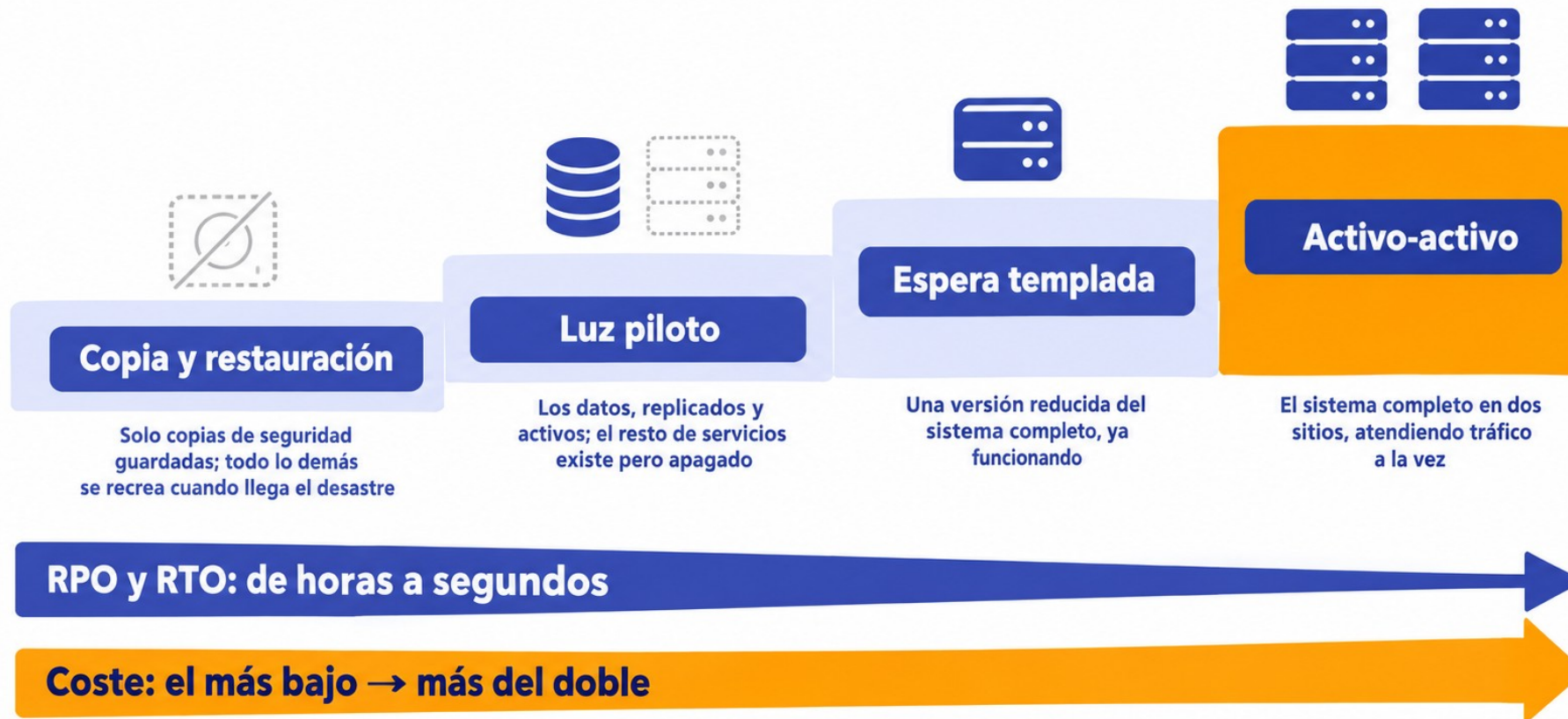
Cuatro estrategias de recuperación ante desastres

AWS describe cuatro estrategias, de menos a más coste. Cada una fija un orden de magnitud distinto de RTO y RPO.

Estrategia	Qué mantienes preparado	RPO y RTO	Coste
Copia y restauración (backup and restore)	Solo copias de seguridad; todo lo demás se recrea cuando llega el desastre	Horas	El más bajo
Luz piloto (pilot light)	Los datos replicados y activos; el resto de servicios existe pero apagado	Decenas de minutos	Bajo
Espera templada (warm standby)	Una versión reducida del sistema completo, ya funcionando	Minutos	Medio
Activo-activo (multi-site)	El sistema completo en dos sitios, atendiendo tráfico a la vez	Segundos o casi cero	El más alto, más del doble

Cuatro estrategias, de menos a más coste

Cuatro estrategias, de menos a más coste



Zona o región: qué protege cada estrategia

Estas estrategias valen tanto para proteger frente a la caída de una zona como frente a la de una región entera.

Multi-AZ, que ya conoces

Es una forma de recuperación dentro de la misma región.

Una segunda región

Sería lo único que protegería frente a la caída de una región entera.

Tu Learner Lab tiene la región fija

Esa estrategia no puedes desplegarla: la razones sobre el papel.

Recomendaciones automáticas de optimización

AWS analiza el uso real de tus recursos y genera recomendaciones a través de dos herramientas.

Trusted Advisor

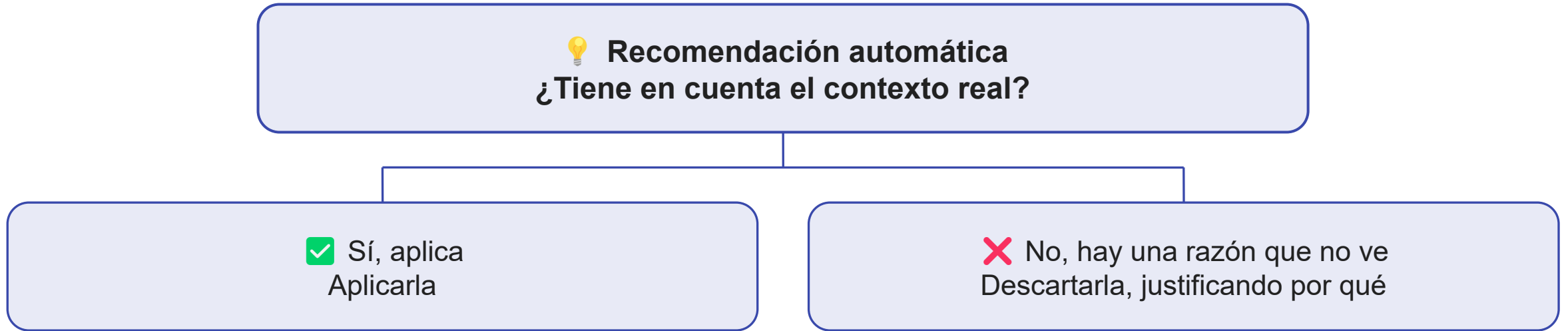
Revisa tu cuenta en conjunto y agrupa sus comprobaciones en categorías muy parecidas a los pilares: coste, rendimiento, seguridad, tolerancia a fallos.

Compute Optimizer

Se centra en si el tamaño de tus instancias encaja con su uso de CPU, memoria y red. Necesita días de métricas para opinar.

Por ejemplo, pueden avisarte de que una instancia lleva semanas con la CPU muy por debajo de su capacidad y podría reducirse de tamaño.

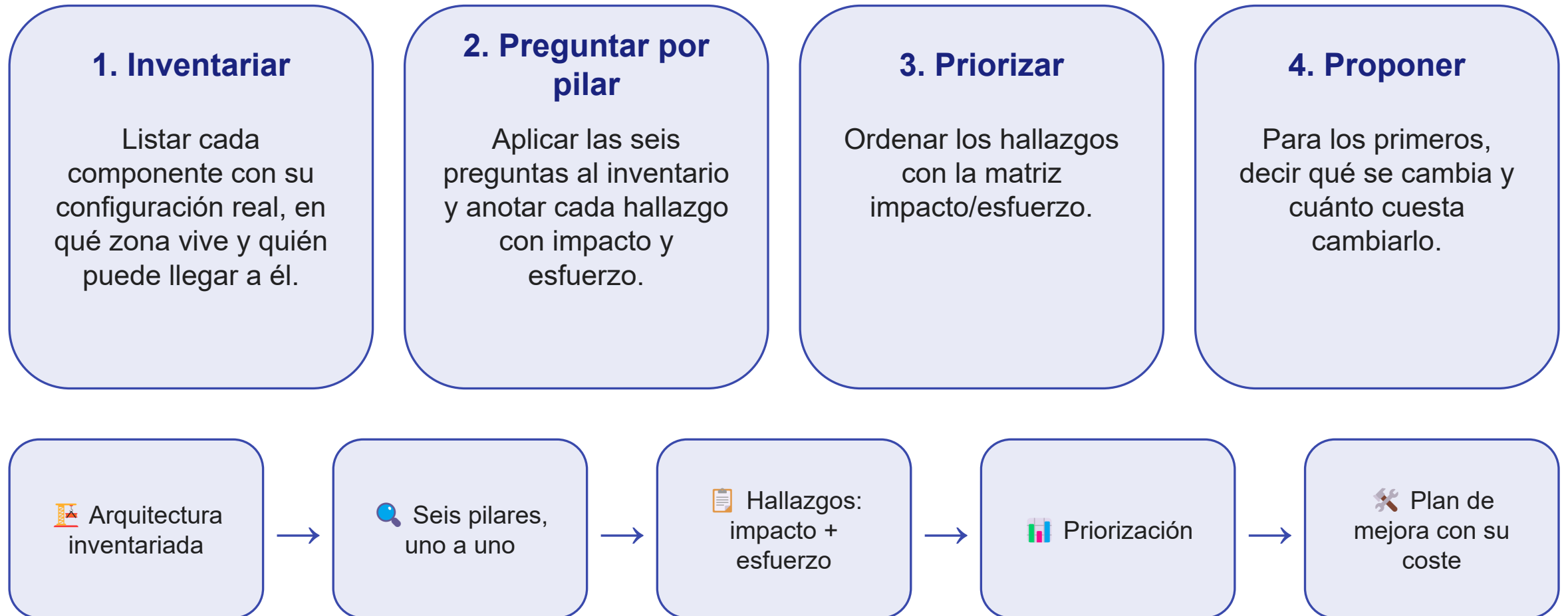
Contrasta cada recomendación antes de aplicarla



Una recomendación puede ser técnicamente correcta y prácticamente errónea

Una aplicación Java puede mostrar la CPU al 10 % durante semanas y, aun así, necesitar la memoria de la instancia que tiene: reducir el tamaño basándose solo en la CPU la dejaría sin memoria. Otras veces dos recomendaciones se contradicen. Interpretar una recomendación es contrastarla con lo que sabes del sistema, no aplicarla porque lo dice una herramienta.

Cómo se audita, en la práctica



Auditar no es repasar cada pilar en abstracto: es recorrer una arquitectura concreta con un método fijo.

Priorizar: impacto frente a esfuerzo

La priorización cruza los dos datos del hallazgo.

	Esfuerzo bajo	Esfuerzo alto
Impacto alto	Se corrige primero	Se planifica
Impacto bajo	Se corrige si sobra tiempo	Se descarta o se acepta

Aceptar el riesgo, por escrito, también es una respuesta válida

Un hallazgo no siempre se corrige

A veces se acepta el riesgo: se deja como está, pero por escrito. Tiene que quedar dicho:



Qué riesgo es



Por qué no se corrige



Hasta cuándo

Es la respuesta correcta cuando corregirlo cuesta más que el daño posible, o cuando no depende de ti, como un servicio que gestiona otro equipo y que tú no puedes cambiar.

AWS Well-Architected Tool

Una herramienta gratuita de AWS

Hace estas mismas preguntas en un cuestionario y devuelve una lista de riesgos.

Hoy no la usas

Haces la revisión a mano para ver qué hay dentro de ese cuestionario.

Con esto ya tienes las piezas para la Actividad 7.1 — Auditoría y mejora, la actividad final del módulo.

Ideas clave

1

Los seis pilares son seis preguntas para revisar lo que ya tienes, no conceptos nuevos.

2

Un hallazgo se anota con su impacto y su esfuerzo. Si un pilar no tiene hallazgos, se anota qué se ha comprobado.

3

Mejorar un pilar suele costar en otro: auditar es decidir qué se prioriza en cada caso, no corregirlo todo.

4

En serie se multiplican las disponibilidades; en paralelo solo cae si caen todas. Un sistema es tan disponible como su eslabón más débil.

5

El RTO mide cuánto tiempo caído es aceptable y el RPO cuántos datos recientes puedes perder; los fija el negocio.

6

Las recomendaciones automáticas son un punto de partida: se contrastan con el contexto real y a veces se descartan, justificándolo.

7

Se prioriza por impacto y esfuerzo, y un hallazgo que no se corrige se documenta como riesgo aceptado.